

# PokeNavigation

Informe de correcciones sobre el código inicial y desarrollo de la actividad propuesta

Consumo de API con Retrofit · Bottom Navigation · RecyclerView

Java · XML · Material Components · Retrofit 3 · PokéAPI

Paquete del proyecto: ue.edu.co.pokenavigation · 18 de septiembre de 2026

## 1. Resumen

La aplicación entregada inicialmente no llegaba a mostrar ninguna pantalla: al ejecutarse quedaba en blanco y el menú inferior no respondía. El diagnóstico arrojó **tres problemas**, uno de compilación, uno de lógica y uno de diseño. Corregidos esos puntos, la aplicación queda funcionando exactamente como describe la guía.

Sobre esa base se desarrolló la **actividad propuesta** de la guía: una pantalla de detalle que consume el endpoint `pokemon/{name}` y muestra imagen oficial, altura, peso, experiencia base y tipos, más la opción de guardar el Pokémon en la sección Favoritos.

| Concepto               | Resultado                                                                  |
|------------------------|----------------------------------------------------------------------------|
| Errores corregidos     | 3 (uno impedía compilar, uno impedía que la app funcionara, uno de diseño) |
| Archivos nuevos        | 5 (3 clases Java, 1 layout, 1 dependencia)                                 |
| Archivos modificados   | 8                                                                          |
| Estructura de paquetes | Reorganizada para coincidir con la guía                                    |

## 2. Parte I — Diagnóstico del código inicial

### 2.1 Error crítico: el `BottomNavigationView` nunca se conectaba

Esta era la causa real de que la aplicación no funcionara. En `MainActivity` estaban correctamente escritos los tres métodos que pide la guía en el paso 11 (`obtenerFragment`, `cargarFragment` y `configurarBottomNavigation`), pero **ninguno de ellos llegaba a ejecutarse**: el método `onCreate` solo llamaba a `initObjects()`.

El resultado era que nunca se le asignaba el `listener` al menú inferior y nunca se cargaba un fragmento inicial dentro del `FragmentManager`. Por eso la pantalla aparecía vacía y tocar los íconos del menú no producía ningún efecto.

#### ANTES

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(...);

    initObjects();           // ← se obtiene la referencia a la vista
}                           // pero nunca se configura ni se carga nada
```

#### DESPUÉS

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(...);

    initObjects();
    configurarBottomNavigation(); // ← se conecta el listener del menú

    // Al abrir la app se muestra la pantalla de Inicio
    if (savedInstanceState == null) {
        bottomNavigation.setSelectedItemId(R.id.navigation_home);
    }
}
```

**Por qué se usa `setSelectedItemId` y no se carga el fragmento directamente.** Ese método marca visualmente la opción “Inicio” y además dispara el `listener`, que es quien carga el `HomeFragment`. Así se cumple el punto 2 de la comprobación de la guía

("compruebe que la opción Inicio aparezca seleccionada") con una sola línea y sin duplicar la carga del fragmento. La condición `savedInstanceState == null` evita que se vuelva a cargar al rotar la pantalla.

## 2.2 Error de compilación: paquetes desalineados

El proyecto no compilaba porque las declaraciones `package` de cuatro archivos no coincidían con los `import` que ya estaban escritos en `MainActivity` y en `HomeFragment`.

| Archivo                | Estaba declarado en                    | Se importaba desde                |
|------------------------|----------------------------------------|-----------------------------------|
| HomeFragment.java      | <code>...pokenavigation.data.ui</code> | <code>...pokenavigation.ui</code> |
| FavoritesFragment.java | <code>...pokenavigation.data.ui</code> | <code>...pokenavigation.ui</code> |
| InfoFragment.java      | <code>...pokenavigation.data.ui</code> | <code>...pokenavigation.ui</code> |
| PokemonAdapter.java    | <code>...data.ui.adapter</code>        | <code>...ui.adapter</code>        |

Se resolvió moviendo los cuatro archivos al paquete `ui`, que además es la estructura que muestra la guía en el paso 12: `data` y `ui` como paquetes hermanos, no `ui` anidado dentro de `data`. Se eliminó la carpeta `data/ui` ya vacía.

## 2.3 Diseño: `fragment_favorites.xml` con el texto de plantilla

El layout conservaba el `TextView` que genera Android Studio al crear un fragmento en blanco, con el texto `@string/hello_blank_fragment` ("Hello blank fragment"), en lugar del contenido de la sección Favoritos.

## 2.4 Lo que sí estaba correcto

Conviene dejar constancia de que el resto de la implementación estaba bien resuelta y no se modificó su lógica:

- El permiso `android.permission.INTERNET` declarado en el `AndroidManifest.xml`.
- La `BASE_URL` terminada en `/` y el endpoint `@GET("pokemon")` sin barra inicial.
- `PokemonResponse` como contenedor de `results`, que es lo que evita el error *Expected BEGIN\_ARRAY but was BEGIN\_OBJECT*.
- El `LayoutManager` y el `Adapter` asignados antes de la llamada a la API.
- La cancelación de `currentCall` en `onDestroyView` y las verificaciones con `isAdded()`, que evitan que la respuesta intente escribir sobre vistas ya destruidas.
- El manejo de los tres estados de pantalla: cargando, contenido y error con botón Reintentar.

## 3. Parte II — Desarrollo de la actividad propuesta

**Enunciado de la guía:** “Amplíe la aplicación para que al seleccionar un Pokémon se abra un nuevo Fragment de detalle. Consuma el endpoint `pokemon/{name}` y muestre al menos la imagen oficial, altura, peso, experiencia base y tipos. Agregue una opción para guardar el Pokémon en la sección Favoritos.”

La actividad se abordó respetando la misma arquitectura que ya usaba el proyecto —modelo, servicio, repositorio, fragmento— de modo que el código nuevo se lee igual que el existente y no introduce patrones distintos a los de la guía.

### 3.1 El modelo: `PokemonDetail.java`

El endpoint `pokemon/{name}` devuelve un objeto mucho más grande que el del listado, con datos anidados en varios niveles. Se creó un modelo que declara únicamente los campos que se van a mostrar y que refleja esa estructura con clases internas.

Dos campos del JSON no pueden nombrarse igual en Java —`base_experience` por el guion bajo y `official-artwork` por el guion medio— así que se usa la anotación `@SerializedName` de Gson para indicarle el nombre real de la llave:

```
@SerializedName("base_experience")
private int baseExperience;

private static class Other {
    @SerializedName("official-artwork")
    OfficialArtwork officialArtwork;
}
```

Además, el modelo expone dos métodos de conveniencia para que el fragmento no tenga que navegar el JSON:

- `getImagenOficial()` — recorre `sprites` → `other` → `official-artwork` → `front_default` y devuelve `null` si algún nivel viene vacío, en lugar de lanzar un `NullPointerException`.
- `getTiposTexto()` — recorre el arreglo `types` y devuelve los nombres separados por coma, por ejemplo `grass, poison`.

### 3.2 El endpoint: `PokeApiService` y `PokemonRepository`

Se agregó una segunda operación a la interfaz del servicio. A diferencia del listado, que usa `@Query` para los parámetros `limit` y `offset`, aquí el nombre va dentro de la ruta, por lo que se usa `@Path`:

```
@GET("pokemon/{name}")
Call<PokemonDetail> getPokemonDetail(
    @Path("name") String name
);
```

Y en el repositorio, el método correspondiente que aísla al fragmento de Retrofit:

```
public Call<PokemonDetail> obtenerDetalle(String nombre) {
    return service.getPokemonDetail(nombre);
}
```

### 3.3 La pantalla: `DetailFragment.java` y `fragment_detail.xml`

El layout es un `ConstraintLayout` con tres capas superpuestas, igual que `fragment_home.xml`: el contenido, el indicador de carga y el contenedor de error. El contenido va dentro de un `ScrollView` porque la imagen de 220 dp más los cinco datos no caben en pantallas pequeñas.

| Vista                      | Contenido                     |
|----------------------------|-------------------------------|
| <code>imgPokemon</code>    | Imagen oficial del Pokémon    |
| <code>tvNombre</code>      | Nombre                        |
| <code>tvAltura</code>      | Altura en metros              |
| <code>tvPeso</code>        | Peso en kilogramos            |
| <code>tvExperiencia</code> | Experiencia base              |
| <code>tvTipos</code>       | Tipos separados por coma      |
| <code>btnFavorito</code>   | Guardar o quitar de Favoritos |

La clase `DetailFragment` repite el mismo patrón de `HomeFragment`: constructor con `super(R.layout.fragment_detail)`, obtención de referencias en `onViewCreated`, llamada asíncrona con `enqueue`, los tres métodos `mostrarCargando()`, `mostrarContenido()` y

`mostrarError()`, botón Reintentar y cancelación de la petición en `onDestroyView`.

El Pokémon a consultar llega por argumentos, que es la forma correcta de pasar datos a un fragmento porque sobrevive a la recreación de la pantalla:

```
public static DetailFragment newInstance(String nombre, String url) {
    DetailFragment fragment = new DetailFragment();
    Bundle args = new Bundle();
    args.putString(ARG_NOMBRE, nombre);
    args.putString(ARG_URL, url);
    fragment.setArguments(args);
    return fragment;
}
```

**Conversión de unidades.** La PokéAPI entrega la altura en decímetros y el peso en hectogramos. Por eso ambos valores se dividen entre 10 antes de mostrarse: Bulbasaur llega como `height: 7` y `weight: 69`, y se muestra como 0.7 m y 6.9 kg.

### 3.4 La navegación desde la lista

El `PokemonAdapter` ya tenía implementada la interfaz `OnPokemonClickListener`, y `HomeFragment` ya estaba escuchando el clic: lo único que hacía era mostrar un `Toast` con el nombre. Ese método se reemplazó por la transacción que abre el detalle.

#### ANTES

```
private void mostrarPokemonSeleccionado(Pokemon pokemon) {
    Toast.makeText(requireContext(),
        "Seleccionó: " + pokemon.getName(),
        Toast.LENGTH_SHORT).show();
}
```

#### DESPUÉS

```
private void mostrarPokemonSeleccionado(Pokemon pokemon) {
    requireActivity().getSupportFragmentManager()
        .beginTransaction()
        .replace(R.id.fragmentContainer,
            DetailFragment.newInstance(pokemon.getName(), pokemon.getUrl()))
        .addToBackStack(null)
        .commit();
}
```

Se usa `addToBackStack(null)` para que el botón atrás del dispositivo devuelva al listado. Las transacciones del menú inferior, en cambio, no se agregan a la pila, de modo que atrás no obliga a recorrer todas las pestañas visitadas.

### 3.5 Los favoritos

La guía no especifica el mecanismo de almacenamiento, así que se resolvió de la forma más simple posible: una clase con una sola instancia (*singleton*) que mantiene un `ArrayList` en memoria mientras la aplicación está abierta.

```
public class FavoritosStore {
    private static FavoritosStore instancia;
    private final List<Pokemon> favoritos = new ArrayList<>();

    private FavoritosStore() { }

    public static FavoritosStore getInstance() {
        if (instancia == null) {
            instancia = new FavoritosStore();
        }
        return instancia;
    }
    // agregar(...), quitar(...), esFavorito(...), getFavoritos()
}
```

Al usar una sola instancia, el `DetailFragment` que guarda y el `FavoritosFragment` que muestra trabajan sobre la misma lista sin necesidad de pasarse datos entre ellos.

El botón de la pantalla de detalle funciona como interruptor: consulta `esFavorito()` y cambia su propio texto entre *Guardar en Favoritos* y *Quitar de Favoritos*, confirmando la acción con un `Toast`.

Por su parte, `FavoritosFragment` dejó de ser una pantalla estática: ahora tiene su propio `RecyclerView` que **reutiliza el mismo `PokemonAdapter`** del listado principal, de manera que las tarjetas se ven idénticas y no hizo falta escribir un adaptador nuevo. La lista se refresca en `onResume()`, que es el momento en que la pestaña vuelve a ser visible después de que el usuario guardó algo desde el detalle. Cuando no hay favoritos, se muestra un mensaje en lugar de una pantalla vacía.

### 3.6 La dependencia nueva: Glide

Retrofit descarga y convierte JSON, pero no imágenes. Para mostrar la imagen oficial se agregó Glide, que es la librería estándar en Android para esta tarea: se encarga de la descarga en segundo plano, del caché y de cancelar la petición si la vista desaparece. Se declaró siguiendo el mismo catálogo de versiones que usa el resto del proyecto:

```
// gradle/libs.versions.toml
glide = "4.16.0"
glide = { module = "com.github.bumptech.glide:glide", version.ref = "glide" }

// app/build.gradle.kts
implementation(libs.glide)
```

Con eso, cargar la imagen en el fragmento es una sola línea:

```
Glide.with(this)
    .load(detalle.getImagenOficial())
    .into(imgPokemon);
```

**Importante al abrir el proyecto.** Como se agregó una dependencia, es necesario ejecutar *Sync Now* (o *File → Sync Project with Gradle Files*) la primera vez, para que Gradle descargue Glide. Sin ese paso, los `import` de `com.bumptech.glide` no resuelven.

## 4. Inventario de archivos

| Estado  | Archivo                                                  | Qué se hizo                                                                                                                                       |
|---------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| MODIF.  | MainActivity.java                                        | Se agregó la llamada a <code>configurarBottomNavigation()</code> y la carga de la pantalla inicial.                                               |
| NUEVO   | data/model/PokemonDetail.java                            | Modelo de la respuesta de <code>pokemon/{name}</code> .                                                                                           |
| MODIF.  | data/model/Pokemon.java                                  | Se agregó el constructor <code>(name, url)</code> para poder crear el objeto al guardarlo en favoritos, y el constructor vacío que requiere Gson. |
| MODIF.  | data/remote/PokeApiService.java                          | Nueva operación <code>@GET("pokemon/{name}")</code> con <code>@Path</code> .                                                                      |
| MODIF.  | data/repository/PokemonRepository.java                   | Nuevo método <code>obtenerDetalle(String nombre)</code> .                                                                                         |
| NUEVO   | data/local/FavoritosStore.java                           | Almacenamiento de favoritos en memoria.                                                                                                           |
| NUEVO   | ui/DetailFragment.java                                   | Pantalla de detalle del Pokémon.                                                                                                                  |
| MODIF.  | ui/HomeFragment.java                                     | Se cambió el paquete y el clic ahora abre el detalle en vez de mostrar un Toast.                                                                  |
| MODIF.  | ui/FavoritesFragment.java                                | Se cambió el paquete y se reescribió como lista de favoritos con RecyclerView.                                                                    |
| MODIF.  | ui/InfoFragment.java ,<br>ui/adapter/PokemonAdapter.java | Solo cambio de paquete; la lógica quedó intacta.                                                                                                  |
| BORRADO | data/ui/                                                 | Carpeta antigua, reemplazada por <code>ui/</code> .                                                                                               |
| NUEVO   | res/layout/fragment_detail.xml                           | Layout de la pantalla de detalle.                                                                                                                 |
| MODIF.  | res/layout/fragment_favorites.xml                        | Se reemplazó el texto de plantilla por título, RecyclerView y mensaje de lista vacía.                                                             |
| MODIF.  | gradle/libs.versions.toml ,<br>app/build.gradle.kts      | Se agregó la dependencia de Glide.                                                                                                                |

## 5. Flujo de funcionamiento de la actividad propuesta

| Paso | Componente        | Responsabilidad                                                            |
|------|-------------------|----------------------------------------------------------------------------|
| 1    | PokemonAdapter    | Detecta el clic sobre la tarjeta y avisa al fragmento.                     |
| 2    | HomeFragment      | Abre DetailFragment enviándole el nombre y la URL del Pokémon.             |
| 3    | DetailFragment    | Solicita el detalle al repositorio usando el nombre recibido.              |
| 4    | PokemonRepository | Invoca <code>getPokemonDetail</code> de <code>PokeApiService</code> .      |
| 5    | PokéAPI           | Retorna el objeto JSON del Pokémon solicitado.                             |
| 6    | PokemonDetail     | Gson convierte el JSON y expone imagen, altura, peso, experiencia y tipos. |
| 7    | Glide             | Descarga y muestra la imagen oficial en el ImageView.                      |
| 8    | FavoritosStore    | Guarda o quita el Pokémon de la lista de favoritos.                        |
| 9    | FavoritesFragment | Muestra la lista de favoritos reutilizando PokemonAdapter.                 |

## 6. Comprobación

1. Abrir el proyecto en Android Studio y ejecutar *Sync Now* para que descargue Glide.
2. *Build → Make Project*: debe finalizar con **BUILD SUCCESSFUL**.
3. Ejecutar en el emulador Pixel 8. La aplicación abre en **Inicio**, con esa opción marcada.
4. Esperar a que se muestre la lista de 30 Pokémon.
5. Seleccionar un Pokémon: se abre el detalle con imagen, altura, peso, experiencia base y tipos.
6. Pulsar *Guardar en Favoritos*: aparece el mensaje de confirmación y el botón cambia de texto.
7. Volver atrás e ir a la pestaña **Favoritos**: el Pokémon guardado aparece en la lista.

8. Cambiar entre Inicio, Favoritos e Información para verificar la navegación.
9. Desactivar la red del emulador y pulsar *Reintentar* para comprobar el estado de error.
10. En Logcat, filtrar por `okhttp` para ver las peticiones HTTP y sus códigos de respuesta.

## 7. Decisiones tomadas y limitación conocida

| Decisión                                             | Alternativa considerada                    | Motivo                                                                               |
|------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------|
| Glide para la imagen                                 | Descargar el bitmap manualmente con OkHttp | Una sola línea de código frente a unas quince, y es la práctica habitual en Android. |
| Favoritos en memoria                                 | SharedPreferences o base de datos Room     | La guía no exige persistencia; el singleton mantiene el ejercicio simple y legible.  |
| Reutilizar PokemonAdapter                            | Crear un adaptador propio para favoritos   | Los datos y el diseño de la tarjeta son los mismos; evita duplicar código.           |
| Paquete <code>ui</code> hermano de <code>data</code> | Corregir solo los imports                  | Coincide con la estructura del paso 12 de la guía y con los imports ya escritos.     |

**Limitación conocida:** los favoritos se almacenan en memoria, de modo que la lista se vacía al cerrar la aplicación. Si se requiriera persistencia, el cambio se concentra en `FavoritosStore`: bastaría con serializar la lista a JSON con Gson —que ya está disponible como dependencia de Retrofit— y guardarla en `SharedPreferences`, sin tocar el resto de las clases.